

NOVACCÈS — Contrat API de l'app agent

Endpoints requis par l'application mobile du poste de contrôle (Expo React Native). Référence : scaffold .NET
NovAcces (Clean Architecture) + maquette validée.

Règles valables pour TOUS les endpoints

- En-têtes obligatoires : Authorization: Bearer <jwt> et X-Site-Id (middleware multi-tenant existant), sauf login.
- Rôle JWT **Agent** : accès limité aux endpoints de ce document. Claims requis : role, siteld, matricule.
- Toute décision de fenêtre horaire est calculée côté serveur (REQ-SEC-02) — ne jamais utiliser l'heure du client.
- Codes erreur : 401 jeton expiré (l'app tente /auth/refresh) · 403 rôle insuffisant · 429 rate limit · 409 conflit de resynchronisation.

1. Authentification (Identity + JWT — rien n'existe encore)

POST /api/auth/login **A CREER**

Connexion de l'agent en début de shift. Rate limiting obligatoire (anti brute-force).

Requête — { "matricule": "SG-0417", "motDePasse": "..." } + en-tête X-Site-Id

Réponse — { "accessToken": "<jwt 8h>", "refreshToken": "...", "expiresIn": 28800, "agent": { "matricule", "nom", "role": "Agent" } }

POST /api/auth/refresh **A CREER**

Renouvellement silencieux du jeton sur 401, sans ressaisie du mot de passe.

Requête — { "refreshToken": "..." }

Réponse — Nouvelle paire accessToken + refreshToken. 401 si refresh invalide/révoqué → l'app revient à l'écran de connexion.

POST /api/auth/logout **A CREER**

Fin de shift : révoque le refresh token (terminal partagé entre agents).

Requête — { "refreshToken": "..." }

Réponse — 204 No Content

2. Configuration du poste

GET /api/site/config **A CREER**

Chargé après login : postes du site + paramètres. Évite de coder ces valeurs en dur dans l'app.

Réponse — { "siteLabel": "SICOPA – Vridi", "postes": [{ "id", "nom" }], "params": { "fenetreAvantMin": 20, "fenetreApresMin": 15, "ttlListeLocaleHeures": 4 } }

3. Scan (le cœur du système)

POST /api/scan **EXISTE**

Scan d'un QR au poste. La logique complète (fenêtre, anti-rejeu, exclusion, cycle entrée/sortie) existe et est testée (Visit.Scan + ScanQrHandler).

Requête — { "signedQrPayload": "<JWS>", "direction": "Entry" | "Exit", "agentId": "SG-0417" }

Réponse — { "isGranted", "isCheckOut", "isSecurityEvent", "verdictCode", "visitorName", "overstayMinutes" }

À compléter : 1) exiger le Bearer JWT (endpoint ouvert aujourd'hui) · 2) ajouter "checkpointId" à la requête pour tracer le poste dans le journal. verdictCode : GRANTED, CHECKED_OUT, INVALID_SIGNATURE, DENIED_Excluded, DENIED_SuspectedDuplicate, DENIED_CycleAlreadyClosed, DENIED_AlreadyConsumed, DENIED_TooEarly, DENIED_TooLate, DENIED_NonBusinessDay, DENIED_Revoked, DENIED_NoActiveEntry.

POST /api/scan/sync **A CREER**

Resynchronisation des scans validés hors ligne, à la reconnexion. ScanQrHandler est déjà conçu pour être appelé avec `IsDegradedMode=true` — il manque l'endpoint batch.

Requête — [{ "signedQrPayload", "direction", "agentId", "scannedAtUtc", "offlineVerdict" }]

Réponse — { "accepted": 3, "conflicts": [{ "visitId", "raison" }] } — ex. QR révoqué pendant la coupure mais validé localement → événement de sécurité (REQ-SEC-06).

4. Mode dégradé (REQ-SEC-06)

GET /api/offline-list **A CREER**

Liste du jour signée ES256 (JWS), téléchargée périodiquement (~15 min) et stockée en SQLite par l'app. La signature et `VerifyDailyOfflineList` existent déjà (`IQrSigningService`) — il manque l'endpoint qui génère et sert la liste. Sert aussi l'écran « Attendus aujourd'hui » (données minimales : ni motif, ni coordonnées).

Réponse — { "generatedAtUtc", "expiresAtUtc" (TTL <= 4 h), "visits": [{ "visitId", "nom", "mode", "fenetreDebut", "fenetreFin", "statut", "present" }] } — le tout enveloppé dans un JWS.

GET /api/keys/public **A CREER**

Clé publique ES256 courante (+ kid) pour la rotation sans republier l'app. La clé est aussi embarquée dans le build (vérification hors ligne des signatures).

Réponse — { "kid": "2026-07", "publicKeyPem": "..." }

5. Temps réel et supervision

WS /hubs/scan-events (SignalR) **EXISTE**

Hub existant (`ScanEventsHub`). L'app s'y abonne pour recevoir `VisitRevoked` / `VisitCreated` et rafraîchir sa liste locale immédiatement, sans attendre le prochain téléchargement. Côté mobile : `@microsoft/signalr`. Optionnel mais recommandé.

GET /api/health **A CREER**

Ping léger pour la pastille EN LIGNE / HORS LIGNE et la bascule automatique en mode dégradé (« réseau présent » ne signifie pas « serveur joignable »).

Réponse — 200 { "status": "ok", "serverTimeUtc": "..." }

Ordre de développement conseillé

1	/api/auth/login + /refresh + sécurisation de /api/scan — débloque le parcours complet en ligne
2	/api/site/config + /api/health — triviaux, finalisent l'app en mode nominal
3	/api/offline-list + /api/keys/public — active le mode dégradé côté app
4	/api/scan/sync — boucle la resynchronisation (conflits) · puis SignalR en option

Bilan : 2 briques existantes (/api/scan à sécuriser, hub SignalR) · 8 endpoints à créer. Le payload QR réel est un JWS signé (visitId + expiration, aucune donnée personnelle — REQ-SEC-01).